

Collection

81

Data Structure

Space for data in memory

Array

array data type

data type

Size

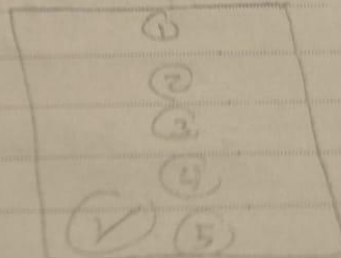
data sequence array

memory

Integer 5 Size

- Add
- Remove
- update
- Select

sequence



index 0 to 499

0 → 499

Size - 1 = array

add 1st index

④ index ③ value 1st index ←

2nd 3rd 4th 5th

0 0(1)

2nd 3rd 4th 5th array 1st 2nd 3rd 4th 5th

5	0
10	1
15	2
20	3
30	4

① index ③ value 1st index ←

Shifts 1st 2nd 3rd 4th 5th

5	0
10	1
15	2
20	3
30	4

Steps of Shifts

← (n+1) → add 1st index

0 0(n)

add + shifts 1st index

add 1st index 1st 2nd 3rd 4th 5th

0(n) 1st 2nd 3rd 4th 5th

0(1) < time 0(n)

Remove → index 1 → 0(n)

→ index 4 → 0(1)

5	0
10	1
15	2
20	3
25	4

update $\rightarrow$ index 2 by 55

So $O(1)$
array 5 10 55 20 25

5
10
55
20
25

Select $\rightarrow$ index 3 $\rightarrow$ 20

So $O(1)$

Array is Performance Time Space $\rightarrow$ So

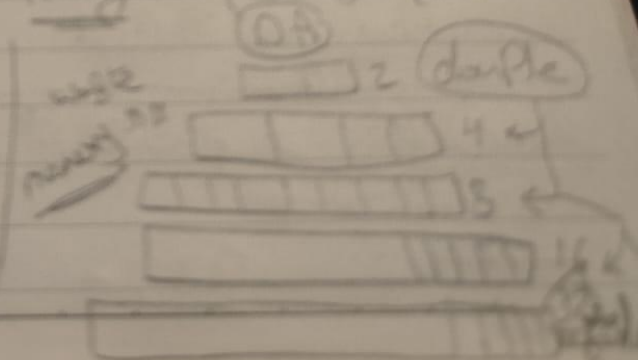
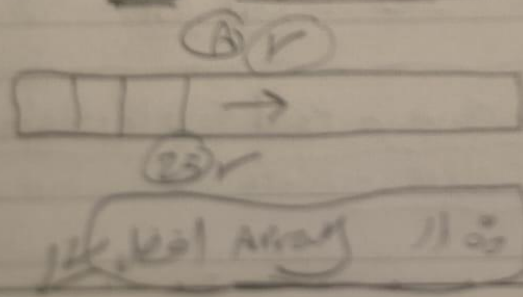
update $\rightarrow$ Select

Array is $O(1)$

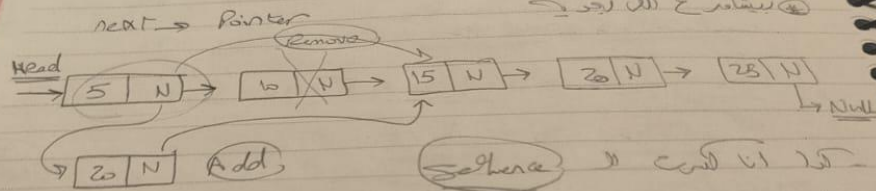
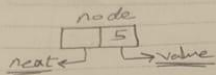
- ① Fixed size
- ② Add $\rightarrow O(n)$
- ③ Remove $\rightarrow O(n)$

Size Dynamic Array is variable

Q. School is having 25 students in the class
Q.1. Data? Dynamic Array is Array is variable



Linked list :-



Add $\rightarrow$ 20 After 5 $\rightarrow$ one step $O(1)$

Remove $\rightarrow$ 10 $\rightarrow$ one step $O(1)$

update $\rightarrow O(n)$

Search $\rightarrow O(n)$

linked list can be (1) linked
(2) indexed $\rightarrow$ array
(3) self linked $\rightarrow$ circular

update $\rightarrow$ search $\rightarrow$ Block Array $\rightarrow$ Sequence $\rightarrow$ 20

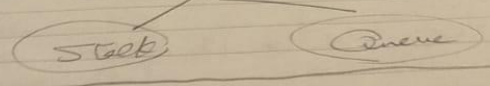
Remove & Add $\rightarrow$ P-Linked list $\rightarrow$ 20

linked list $\rightarrow$ 20 $\rightarrow$ Remove, Add $\rightarrow$ System $\rightarrow$ 20

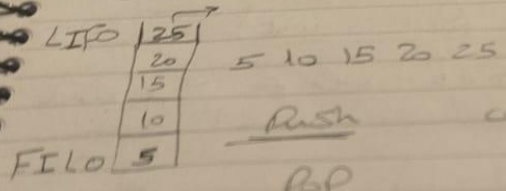
Array $\rightarrow$ " " " Search $\rightarrow$ 20 " " " " " "

V.I.

2 Data Structure



1) Stack



Push add

Pop

اخراج

من الذاكرة

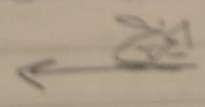
⇒ 25 → 20 → 15 → 10 → 5

LIFO

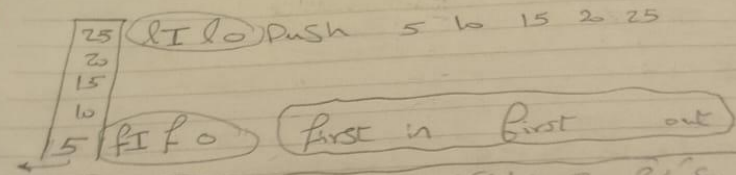
or

FILO

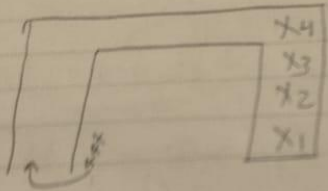
Last in first out
اول ما ادخل اول ما يخرج



Queue



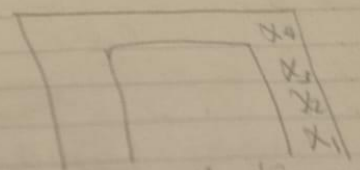
Set of elements which are used to store data



Stack is a linear data structure which follows the Last In First Out (LIFO) principle.

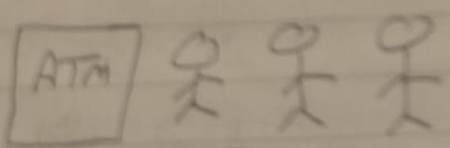
LIFO

is stack



Queue is a linear data structure which follows the First In First Out (FIFO) principle.

FIFO
is Queue



FIFO
is Queue

Ans

map Value & Key 87

Key	Value
id ₀₅	= 54
id ₁	= 60
id ₃	= 90
id ₄	= 20

Key	Value
Egy	5
SP	10
SA	15
SR	20

COVID

map

Key String

Value int

data type

COVID → Egy → 5
→ Remove by Key

unique as Key
لا يمكن أن يكون

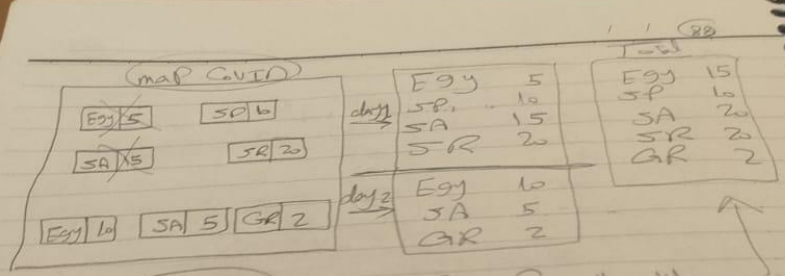
K	V	K	V
Egy	5	SP	10
SA	15	SR	20
Egy	5		

Override [Egy 20] ليست له قيمة

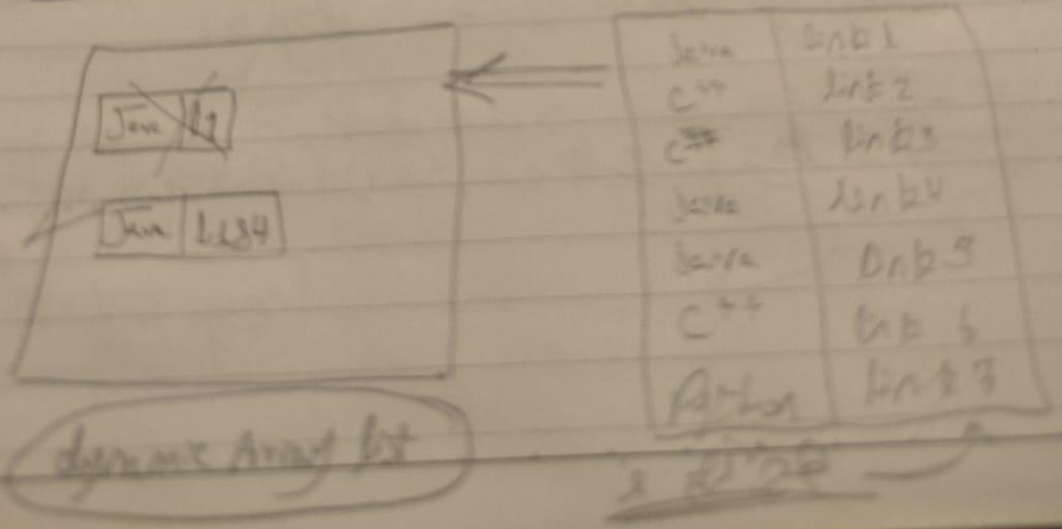
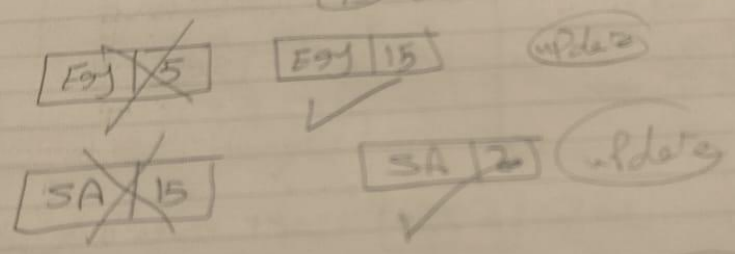
Egy 30 → لا يمكن ~~Egy 5~~

Value & Key Said map لا يمكن أن يكون لا يمكن أن يكون

علاقة الـ map مع الـ data



IS Total map update
 (Egy) → P key map update
 (15) → 5 → 15



Array

`int ids[] = new int[5];`

`ids[0] = 43;`

`ids[1] = 28;`

`ids[2] = 44;`

`ids[3] = 53;`

`ids[4] = 24;`

`System.out.println(ids[3]);`

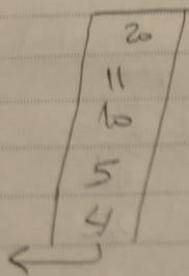
// 53

ArrayList → dynamic array

Priority Queue →

10 5 11 4 20

Push



Queue 1

Array Deque →

Queue of stack

Queue 2

Vector

import → java.util.Vector

is class available in main

Similar to ArrayList // of Vector

① Synchronized → thread-safe → performance is low as it is synchronized

② Full → automatically doubles its capacity...

Vector

30 40 50 60 5 10 15 20

Thread 1

Thread 2

5

30

10

40

15

50

20

60

all cells are

ArrayList

5 30 40 10 50 15

T1

T2

5

30

10

40

15

50

20

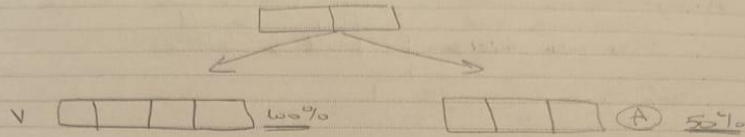
60

insert

capacity is

80 after

① Synchronized
data is safe ←



Priority Queue :-

Priority Queue is a queue in which elements are stored in the order of their priority.

(Comparator.reverseOrder()) →

Priority Queue

Array Deque :-

(Array-Double-Ended Queue) / Queue

Queue or Stack (Deque)

Priority Queue is a queue in which elements are stored in the order of their priority.

Array Deque & Stack are both used for storing elements.

Synchronized → ArrayList & Vector & LinkedList

Not Synchronized → Stack & Deque ← Array Deque

Array Dele → Stack
Push → add
Pop → delete

Array Dele → Queue
offer → add
Poll → delete

Set :-

↳ dynamic Array

5 | 10 | 5 | 10 | 20
Array List

5 | 10 | 20
SET

(duplication) keeno unique or Set

(duplication) keeno unique or Set

HashSet :-

unordered Set →

Set

hashSet.add("Etem")
hashSet.add("Ahmed")
hashSet.add("Sara")

نوع القيع باي ترتيب

System.out.println(hashSet);

Sara
Etem
Ahmed

نوع

نوع

very fast & null safe value ← hashset

linked hashset :-

ordered

map

size

Islam
Ahmed
Sara

value & key

✓ Slower than hashset

value & key ←

logic & value & key

map :-

Value & key value & key create value
While unique value key & value

Add/Remove/Size/Update in class

one step

(old)

new

(4)

Value & key map & key

[0]	[13]
[2]	[3]
[4]	[5]
[7]	[8]

Array of buckets
value & key

map
key & value
buckets
key & value

("Eslam", 25)
key value

1) bucket 2) key value Hash function

unique key value

hash("Eslam") → 5

key value
hash value

hash("Ahmed") → 5

no duplicates / override

Eslam) hash key value ("Eslam")

bucket value

1. 100
(V.D) - (map) » mechanism

hashmap » based on buckets
function » if it is not in the bucket
bucket » search | delete | Add | update »
... (1) ... direct

① Hash map » implement map

- non-synchronized → not thread-safe...
- ... (1) ... thread
- Allow null → key
- Order » ...
- very fast.

② Hash table

... control ... hashmap

synchronized ← hash table

key » null » ...

② Slower than Hash map not synchronized

Linked Hashmap -
non synchronized → like HashMap

Allow null → like HashMap

insertion will not be linked to its previous node
maintain order

Linked Hashmap is not HashMap

Class Linked Hashmap extends HashMap

→ also implements map

Class HashTable implements map

Subclass ←
Synchronized version

names.set(1, "Sara");
System.out.println(names);

LinkedList<String> names = new LinkedList<>();
names.add("ahmed");
names.add("Osama");

Vector<String> vector = new Vector<>();

Stack<String> stack = new Stack<>();

stack.push(item: "ahmed");
stack.push(item: "ali");
stack.push(item: "Osama");
stack.push(item: "Sara");

System.out.println(stack);

stack.pop();

System.out.println(stack);